

TIVIDY

The team

Tamlyn Khan (u24675815)
Yoshua Smit (u25433726)
Isabelle Engelbrecht (u25090501)
Jonathan Witter (u25108582)
Sudud Abughazaleh (u25282728)

Github Repo Link:

<https://github.com/IridescentDucky/COS214-Practical-6-Project>

Workflow-management research

- **What is a workflow and what is a workflow management system?**

A workflow is an automated process where tasks pass between people according to set rules. A WfMS stores, runs and manages these processes (Hollingsworth, 1995; WfMC, 1996). So TIVIDY's engine must run processes without containing them.

- **What is the difference between a workflow definition and a running workflow instance?**

A definition is the template of steps, order and roles; an instance is one run of it, with its own data and history (WfMC, 1996). So these need separate classes, and changing one instance must not affect the others.

- **How are activities or work items represented?**

Activities create work items that go to a worklist and move through states such as waiting, active and completed (WfMC, 1996). So work items need a state-based lifecycle.

- **How are work items assigned and routed?**

Routing uses sequence, parallel, choice and loops, and each product supports a different subset (van der Aalst et al., 2003). Work is either pushed to someone by role or workload, or pulled from a shared list (Russell et al., 2005). So both routing and assignment must be swappable, not hard-coded.

- **How are approvals, rejection and escalation handled?**

An approval is a work item whose result decides the next step. Rejection sends work back, and escalation reassigns it when a deadline passes (WfMC, 1996; Russell et al., 2005). So deadlines and escalation rules must be configurable.

- **How are participants, roles and responsibilities represented?**

Definitions name roles, not people, and the engine picks a person at run time from the organisation's structure (WfMC, 1996). So staff changes should not require changing definitions.

- **How are dependencies, events and failures handled?**

Steps wait on conditions or joins, events trigger actions, and each type of failure gets a chosen response such as retry, skip or undo (WfMC, 1996; Russell, van der Aalst and ter Hofstede, 2006). So many parts must be able to react to one event, and failure handling must be configurable.

- **How do workflow systems interact with external systems?**

Engines reach outside systems through a separate interface (Hollingsworth, 1995); Camunda, for example, queues jobs for external workers to pick up (Camunda, n.d.b). So TIVIDY needs a translation layer for each external system.

- **How is workflow history or audit information recorded?**

Every state change is logged with a timestamp (WfMC, 1996). So actions should be stored as history objects, which also makes undo possible.

- **How are running workflows monitored and reported on?**

Monitoring is separate from execution, and logs are analysed afterwards to find bottlenecks (Hollingsworth, 1995; van der Aalst, 2016). So reporting should read the history from outside the core classes.

- **What makes a workflow engine reusable across different processes?**

All process rules live in the definition, not the engine (WfMC, 1996). So no scenario-specific code belongs in the engine.

- **Which information belongs to a workflow definition and which belongs to a running instance?**

Definition: steps, order, conditions, roles and deadlines. Instance: current state, data, assigned people and history (WfMC, 1996).

- **How do real workflow systems model nested activities or sub-processes?**

A sub-process has its own definition and can be nested inside others (WfMC, 1996). So nested work forms a tree the engine can traverse.

- **Which decisions are normally configurable rather than hard-coded into the workflow engine?**

Routing, assignment, escalation and failure handling (WfMC, 1996; Russell et al., 2005; Russell, van der Aalst and ter Hofstede, 2006).

- **How are changes to workflow definitions handled while work is already in progress?**

A change creates a new version, and running instances stay on the old one unless they can be safely migrated (Camunda, n.d.a; Rinderle, Reichert and Dadam, 2004). So definitions must be versioned.

References

Camunda (n.d.a) Process versioning. Available at: <https://docs.camunda.org/manual/latest/user-guide/process-engine/process-versioning> (Accessed: 1 October 2026).

Camunda (n.d.b) *External tasks*. Available at: <https://docs.camunda.org/manual/latest/user-guide/process-engine/external-tasks> (Accessed: 1 October 2026).

Hollingsworth, D. (1995) *The workflow reference model*. Brussels: Workflow Management Coalition.

Rinderle, S., Reichert, M. and Dadam, P. (2004) 'Correctness criteria for dynamic changes in workflow systems: a survey', *Data & Knowledge Engineering*, 50(1), pp. 9–34.

Russell, N., van der Aalst, W.M.P. and ter Hofstede, A.H.M. (2006) 'Workflow exception patterns', in *CAiSE 2006*. Berlin: Springer, pp. 288–302.

Russell, N., van der Aalst, W.M.P., ter Hofstede, A.H.M. and Edmond, D. (2005) 'Workflow resource patterns', in *CAiSE 2005*. Berlin: Springer, pp. 216–232.

van der Aalst, W.M.P. (2016) *Process mining: data science in action*. 2nd edn. Berlin: Springer.

van der Aalst, W.M.P., ter Hofstede, A.H.M., Kiepuszewski, B. and Barros, A.P. (2003) 'Workflow patterns', *Distributed and Parallel Databases*, 14(1), pp. 5–51.

Workflow Management Coalition (WfMC) (1996) *Terminology & glossary*. Brussels: Workflow Management Coalition.

Scenario description

We have decided to implement an Exam Management System (EMS) using the TIVIDY (the workflow-management engine) that we have created. Our system governs the end-to-end lifecycle of exam papers, prioritising strict paper tracking and secure state management.

The system makes use of the Abstract Factory design pattern to create the blueprints for the papers that the students will write. We then make use of the Prototype pattern to make clones of the paper so that each student can have their own individual paper (this simulates printing papers). Once the paper is written by the student, it has a variety of states that it goes through, starting with a *WrittenState*. The Observer pattern is used to monitor when the paper has been delivered to the markers, automatically sending a notification so the system knows to change the paper's state. From there, markers will be able to set the paper into a *MarkingState*, a *PausedState* (which allows them to safely pause marking and continue at a later time without losing data), a *MarkedState*, and finally a *CapturedState*. The markers also make use of the Chain of Responsibility pattern to have a clear hierarchy between tutors, ALs, lecturers, and module coordinators. This dictates who gets to mark what and handles dynamically escalating the paper up the chain to the next marker when needed.

GoF participant mapping and Pattern Justification

1. Memento

Design problem in TIVIDY:

The system must save and restore a WorkItem's prior state without exposing its internal representation to the object that stores the snapshot.

GoF participant mapping:

- Originator: WorkItem
- Memento: WorkItemMemento
- Caretaker: Caretaker

How participants collaborate:

WorkItem creates a WorkItemMemento through createMemento() and restores it through setMemento(). Caretaker stores the memento but does not inspect its internals. In the EMS scenario, ExamStatisticsOriginator, StatisticsMemento, and StatisticsCaretaker apply the same collaboration to statistics snapshots.

What may vary:

- The state captured in a snapshot
- When snapshots are taken and restored

Why this pattern is appropriate:

TIVIDY can support pause, rollback, or recovery without breaking WorkItem encapsulation.

2. Abstract Factory and Prototype

Design problem in TIVIDY:

Different workflow families require consistent, related workflow-instance products, and an existing workflow template must be reproducible without exposing its construction details.

GoF participant mapping:

- Abstract factory: AbstractWorkflowFactory
- Concrete factories: ConcreteWorkflowFactory1, ConcreteWorkflowFactory2
- Abstract products: AbstractWorkflowInstanceA, AbstractWorkflowInstanceB
- Concrete products: WorkflowInstanceA1, WorkflowInstanceA2, WorkflowInstanceB1, WorkflowInstanceB2
- Prototype: Prototype
- Prototype implementations: AbstractWorkflowInstanceA and AbstractWorkflowInstanceB, through clone()

How participants collaborate:

Clients request a product family from AbstractWorkflowFactory. A concrete factory creates compatible A and B workflow-instance products. The workflow-instance abstractions implement clone() through Prototype so an existing template can be copied when a new independent instance is needed.

What may vary:

- The family of workflow-instance products created for a workflow type
- The concrete template used to create a new instance

Why these patterns are appropriate:

Abstract Factory prevents clients from hard-coding a particular compatible workflow family. Prototype supports creating further instances from an established template. In the EMS scenario, ModulePaperAbstractFactory creates the module-specific paper family and ExamPaperPrototype clones an ExamPaper for a student.

3. Strategy

Design problem in TIVIDY:

Different workflow definitions need different decision algorithms. TIVIDY must choose an assignment, routing, or escalation rule without embedding that algorithm in the mediator.

GoF participant mapping:

- Context: WorkflowMediator
- Strategy: Strategy
- Concrete strategies: ConcreteStrategyA, ConcreteStrategyB, ConcreteStrategyC

How participants collaborate:

WorkflowMediator holds a Strategy reference and delegates algorithm() to the selected concrete strategy. The strategy supplies the decision while the mediator remains responsible for workflow coordination.

What may vary:

- The algorithm used to choose the next workflow action or assignee
- The conditions used for routing or escalation

Why this pattern is appropriate:

The decision rule is a policy that changes independently of workflow coordination. Strategy lets TIVIDY replace that policy without changing WorkflowMediator.

4. State

Design problem in TIVIDY:

A WorkItem behaves differently as it progresses through its lifecycle. Operations valid while work is in progress must not be accepted after it has completed or been archived.

GoF participant mapping:

- Context: WorkItem
- State: WorkItemState
- Concrete states: Created, InProgress, Paused, Completed, Archived

How participants collaborate:

WorkItem stores its current WorkItemState and delegates handleState() to it. The state determines the permitted next() transition and WorkItem updates its state through set(). In the EMS scenario, ExamPaper and PaperState play the same role with exam-specific states such as WrittenState, MarkingState, and CapturedState.

What may vary:

- Legal transitions and state-specific behaviour
- Extra scenario states, such as DeliveredState, ExceptionState, ValidatedState, or FailedState

Why this pattern is appropriate:

State prevents lifecycle checks from becoming a large conditional chain in WorkItem and keeps each state's transition rules together.

5. Composite

Design problem in TIVIDY:

Workflows can contain both individual work items and nested groups of work. Clients should be able to handle both through one interface.

GoF participant mapping:

- Component: WorkItemComponent
- Leaf: WorkItem
- Composite: CompositeWorkItem
- Client: WorkflowMediator

How participants collaborate:

CompositeWorkItem owns children: CompositeWorkItem and provides add(), remove(), getChild(), and operation(). WorkItem is the leaf. WorkflowMediator can invoke the common component operation without needing to distinguish a single item from a group.

What may vary:

- The nesting and grouping of work items
- The number and structure of child work items

Why this pattern is appropriate:

It models stages and sub-processes while preserving a uniform interface for workflow traversal and execution.

6. Decorator

Design problem in TIVIDY:

A work item may need optional responsibilities without creating a subclass for every combination of behaviours.

GoF participant mapping:

- Component: WorkItem
- Decorator: Decorator
- Concrete decorators: ConcreteDecoratorA, ConcreteDecoratorB

How participants collaborate:

Decorator keeps a reference to a WorkItem and exposes operation(). A concrete decorator adds behaviour and then delegates to the wrapped work item. In the EMS scenario, ExamDecorator wraps an ExamComponent, with ModerationDecorator and ConcessionDecorator adding exam-specific behaviour.

What may vary:

- The optional behaviours applied to a particular work item
- The combination and order of added responsibilities

Why this pattern is appropriate:

Decorator layers independent concerns onto a task at runtime while retaining the same component interface.

7. Observer

Design problem in TIVIDY:

Several components may need to react to one workflow event without the workflow coordinator knowing the details of every reaction.

GoF participant mapping:

- Subject: WorkflowMediator
- Concrete subject: ConcreteWorkflowMediator
- Observer: Observer
- Concrete observer: ConcreteObserver

How participants collaborate:

WorkflowMediator maintains observerList, attaches and detaches Observer objects, and calls notify() when a relevant workflow event occurs. ConcreteObserver receives update() and performs its own reaction. In the EMS scenario, ExamDelivery is the subject and Marker is the observer hierarchy.

What may vary:

- The observers registered for a workflow event
- The reactions to completion, delivery, or state change

Why this pattern is appropriate:

Notifications, monitoring, and downstream reactions should be independently extendable rather than hard-coded into the coordinator or work item.

8. Adapter

Design problem in TIVIDY:

The generic workflow engine must use external services without depending on their incompatible interfaces.

GoF participant mapping:

- Client: WorkflowMediator
- Target: Target
- Adapter: Adapter
- Adaptee: Adaptee

How participants collaborate:

WorkflowMediator depends on Target and calls request(). Adapter implements Target, holds an Adaptee, and translates request() into the adaptee's specificRequest().

What may vary:

- The external service or legacy interface

- The translation between a workflow operation and an external API

Why this pattern is appropriate:

Adapter keeps TIVIDY independent of a particular external system while allowing a suitable connector to be introduced or replaced.

9. Mediator

Design problem in TIVIDY:

Workflow objects, state objects, policies, and observers must collaborate without every object depending directly on every other object.

GoF participant mapping:

- Mediator: WorkflowMediator
- Concrete mediator: ConcreteWorkflowMediator
- Colleagues: WorkItem and the other registered workflow colleagues

How participants collaborate:

Colleagues communicate workflow-relevant changes to the mediator.

ConcreteWorkflowMediator coordinates the response: it applies its Strategy, coordinates state-related behaviour, and notifies registered observers. The colleague objects should not coordinate each other directly.

What may vary:

- Which colleagues participate in a workflow
- The coordination rules applied after a state change or workflow event

Why this pattern is appropriate:

TIVIDY is a coordination engine. Mediator centralises collaboration rules and avoids a network of direct dependencies between the reusable workflow components.

10. Chain of Responsibility

Design problem in TIVIDY:

An escalation request may need to pass through a sequence of eligible handlers until one can deal with it.

GoF participant mapping:

- Handler: TaskEscalationManager
- Concrete handlers: ConcreteHandler1, ConcreteHandler2
- Successor link: successor: Handler

How participants collaborate:

The client submits a request to TaskEscalationManager through handleRequest(). A handler processes it when appropriate; otherwise it forwards the request to successor. In the EMS scenario, Marker is the handler hierarchy, with Tutor, AssistantLecturer, Lecturer, and ModuleCoordinator forming the escalation path.

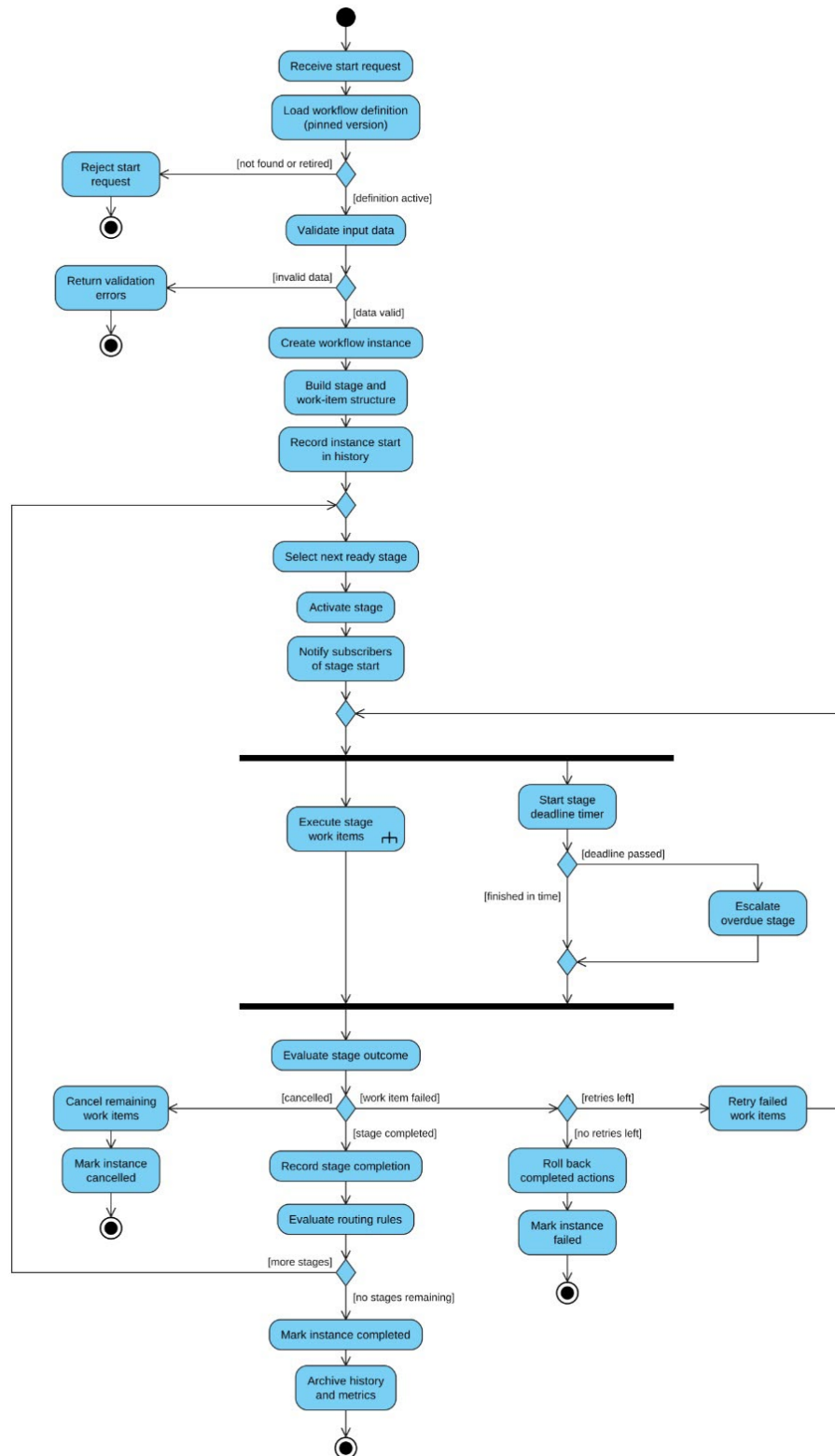
What may vary:

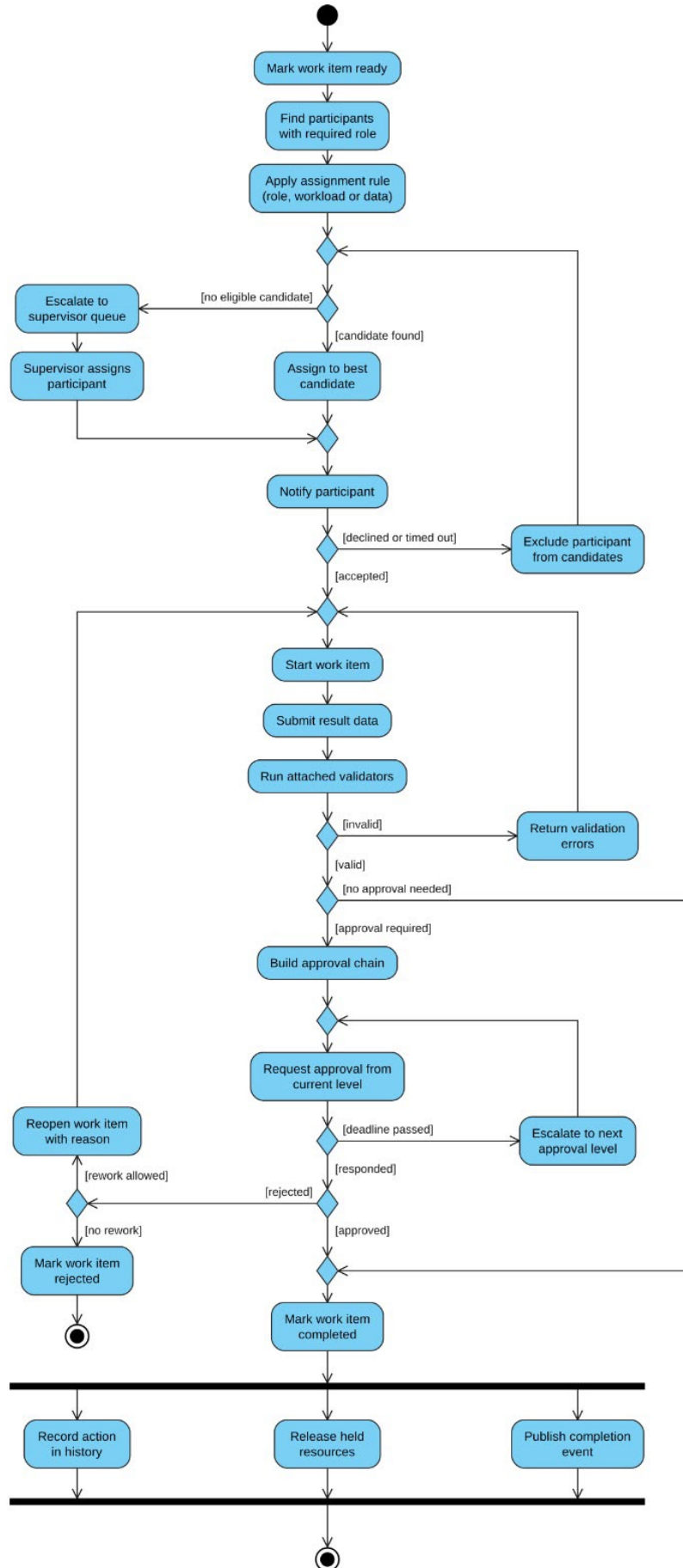
- The handlers included in an escalation path
- Their order and the conditions under which each handles a request

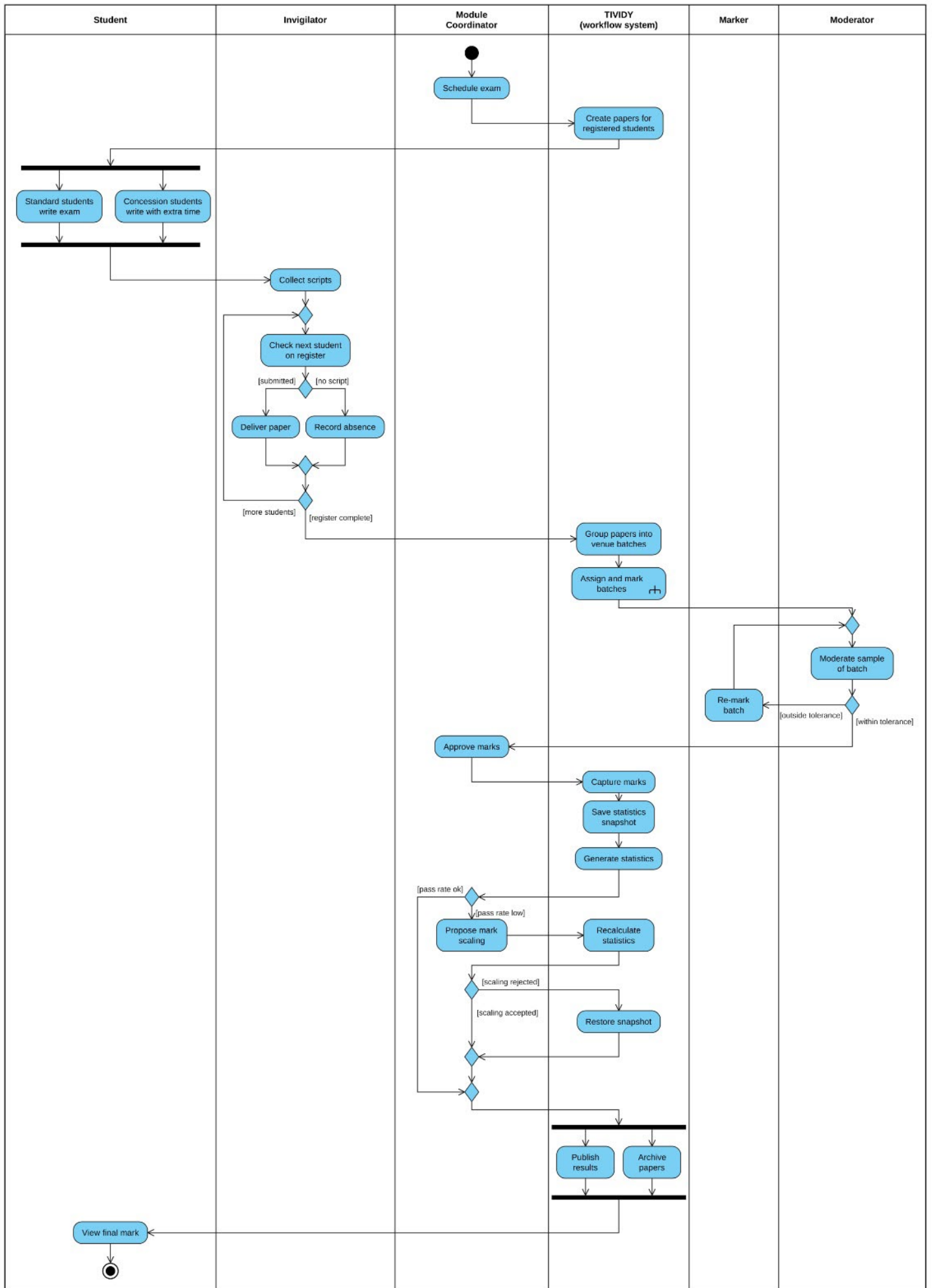
Why this pattern is appropriate:

The source of a workflow event should not need to know which participant will ultimately resolve it. Handlers can be added or reordered without changing the requesting code.

UML Activity Diagrams



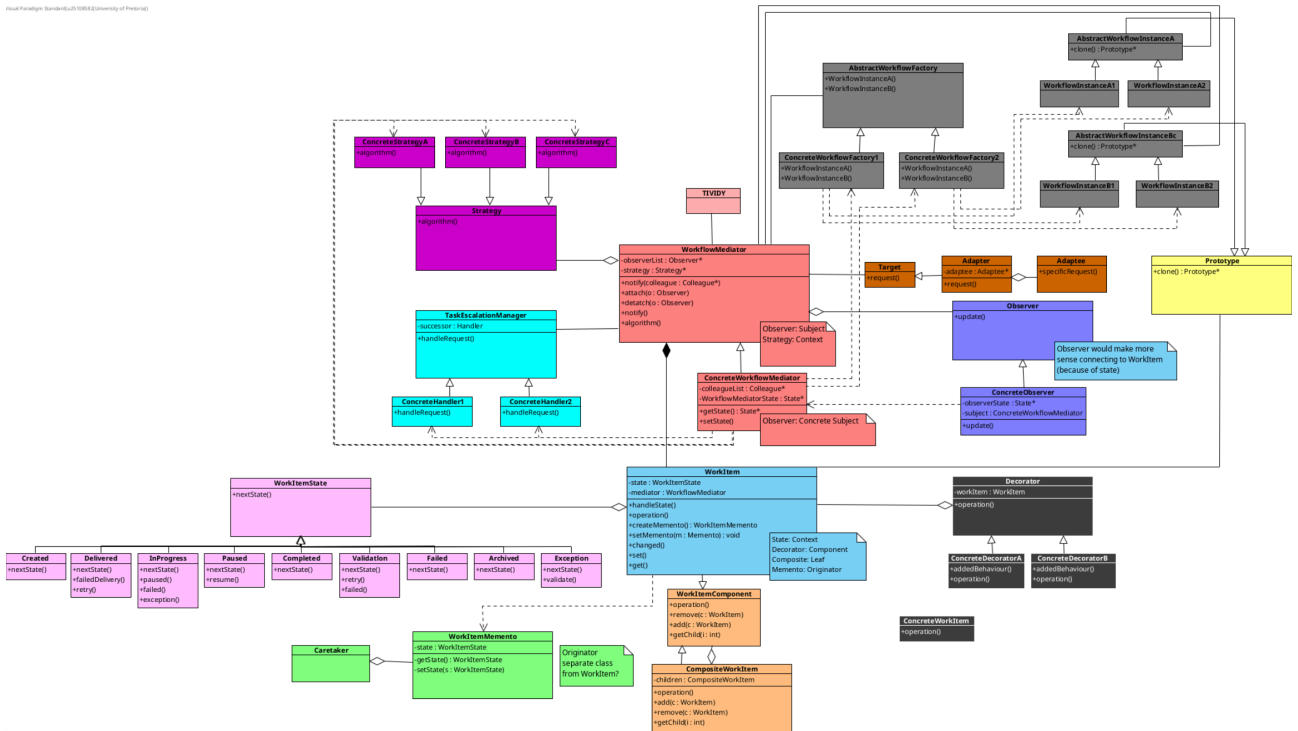




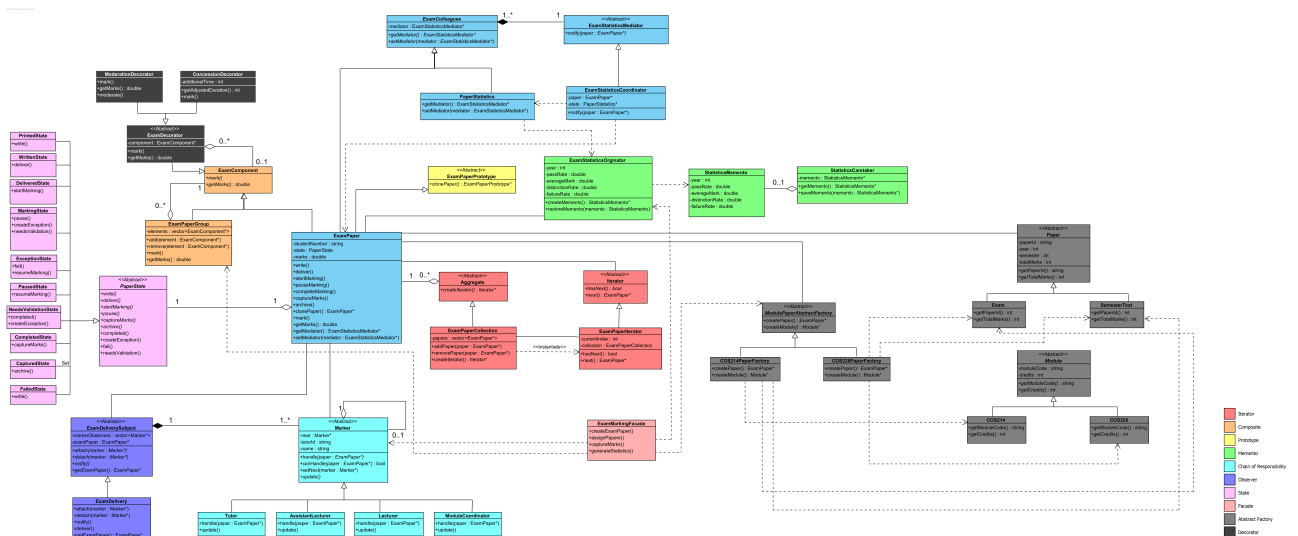
UML Class Diagrams

TIVIDY UML class diagram

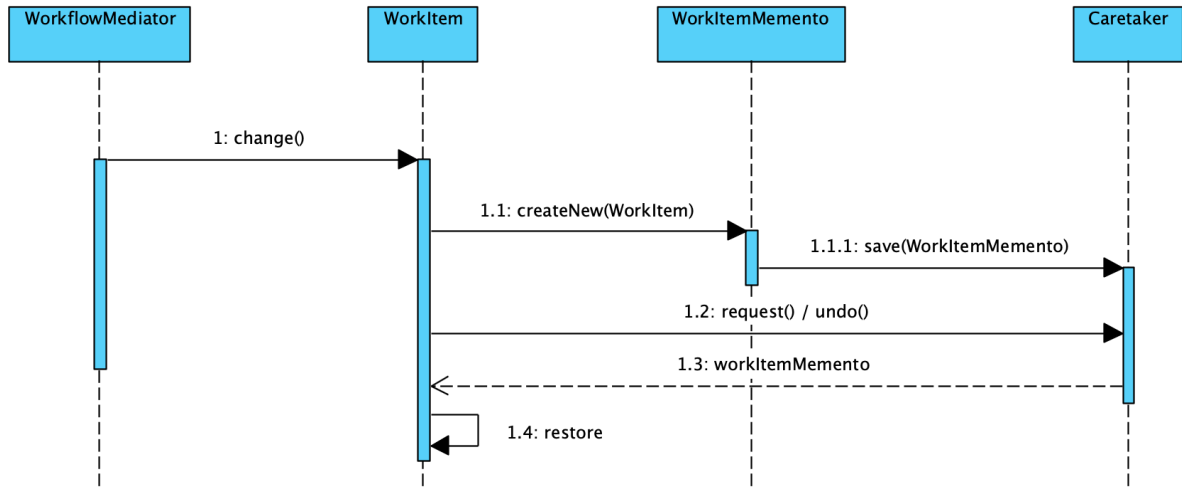
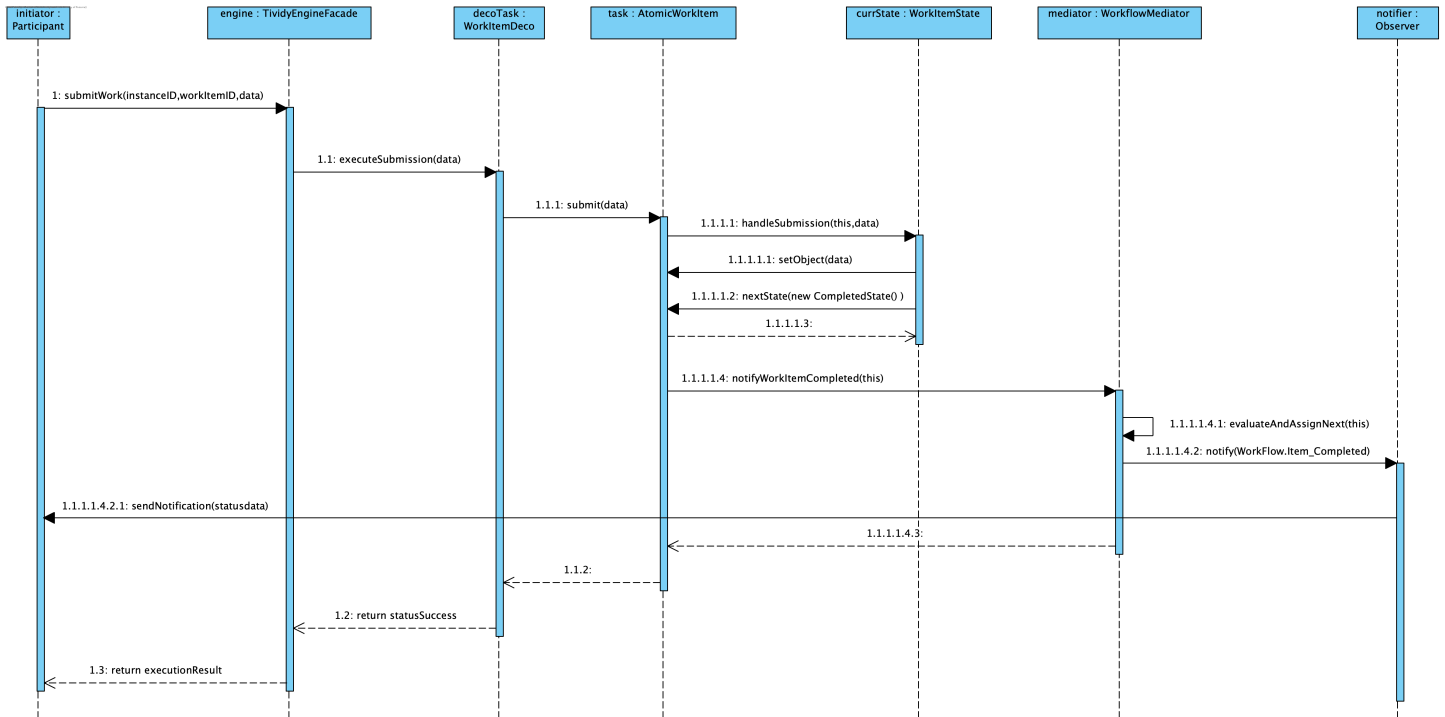
Real Privilege Standard (2018) (University of Pretoria)



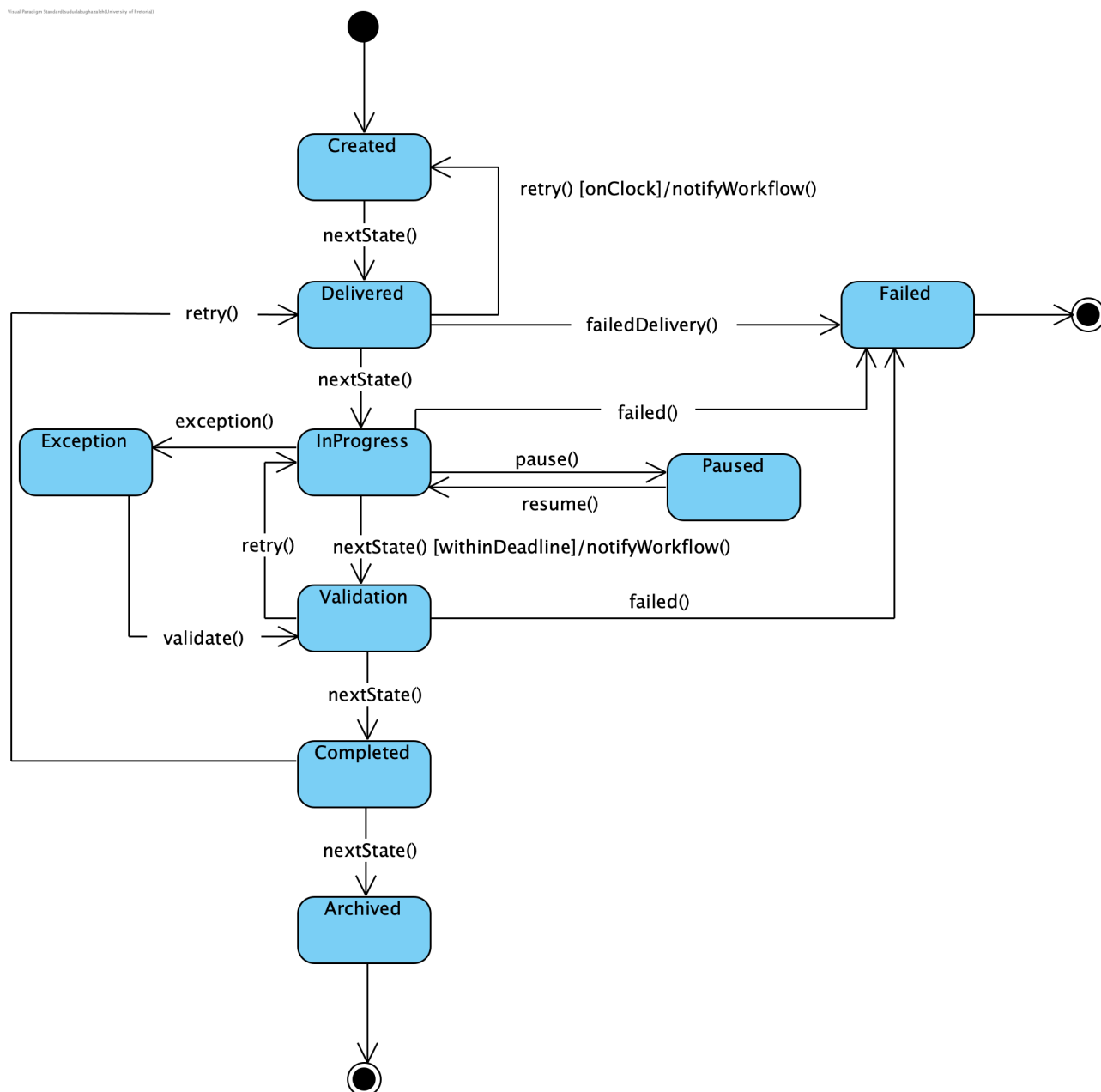
Scenario UML class diagram



UML Sequence Diagrams



UML State Diagram

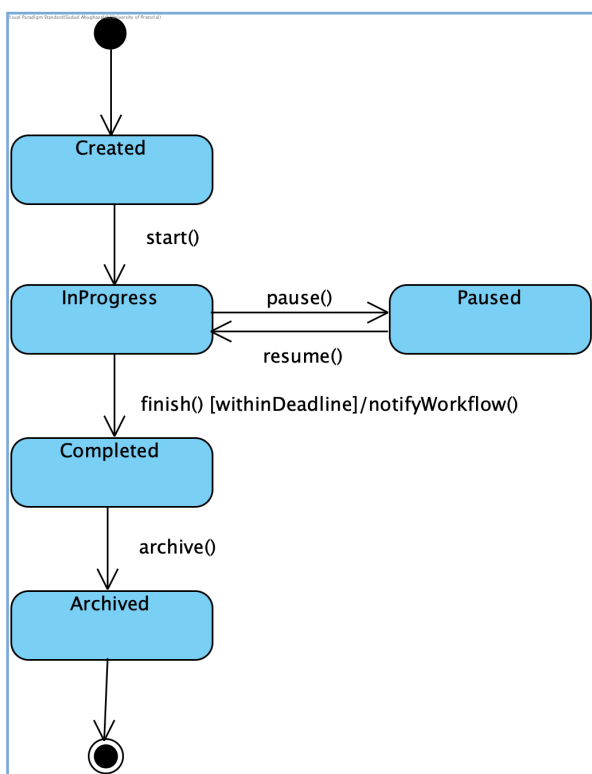


Design decisions and Revision history

Design decisions

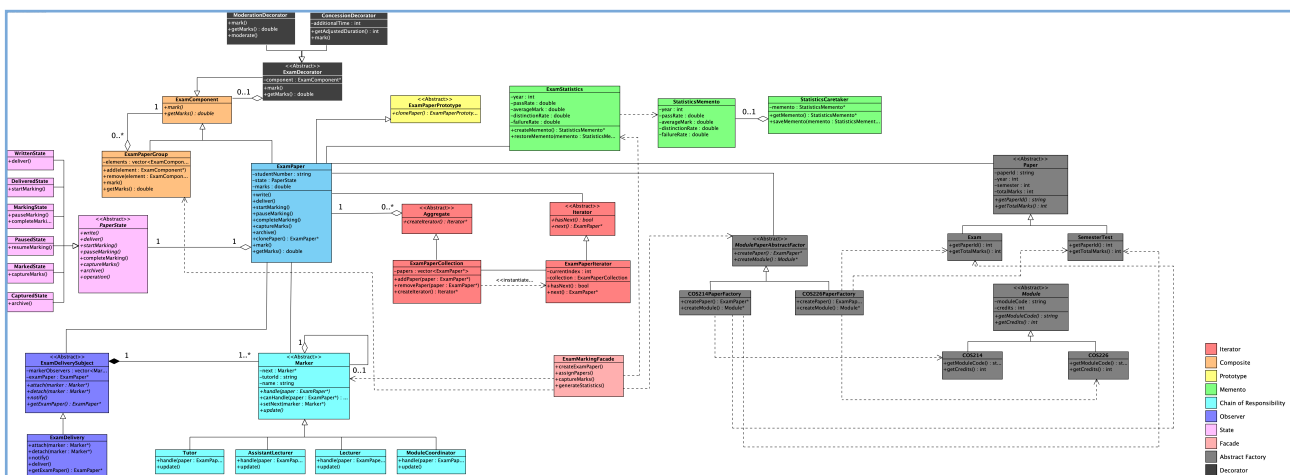
- For our scenario, we decided that we did not want to use the Strategy and Adapter design patterns so we did not include it although it is in TIVIDY. We wanted to use the Iterator design pattern in our scenario so we added that into our class diagram for the scenario.
- We initially really wanted to make use of the Strategy design pattern in our scenario, however we found it really difficult to incorporate it into our scenario in a way that is logical for the system so we decided to not add it.

Revision history



•On the left is an image of our first state diagram. We had consulted with an AL and from there we added more states such as Delivered, Exception, Validation and Failed. We also added a retry function from certain points as well as failed functions in some states. We also made sure that it would be fine to use a function called nextState instead of setting predetermined names for the function that allows the object to move from one state to the next.

The below class diagram was our initial class diagram for our scenario the only changes that we made was that we added the Mediator pattern and some more concreteStates.



Team contribution statement

Tamlyn - Research and three activity diagrams.

Yoshua - Initial system design and design pattern justification.

Isabella - scenario UML class diagram and Git repo.

Jonathan - TIVIDY UML class diagram and sequence diagram.

Sudud - sequence diagram, state diagram, GoF participants and PDF.